

Beginner's Guide: J0604

ALMA Thesis Planner (automated pipeline)

0.1 1. The Goal

Imagine you are standing outside at night looking at a distant streetlamp through fog. You can't see the fog directly — it's too faint to glow on its own — but you can tell it's there because it dims the lamp. And if you had a good enough instrument, you could tell how *fast* the fog was drifting toward you or away from you, because motion very slightly changes the color of the light that passes through it.

That is exactly what this project does, except the "streetlamp" is an entire galaxy, and the "fog" is gas floating in front of it.

The galaxies here are **starburst galaxies** — galaxies that are building new stars hundreds or thousands of times faster than our own Milky Way does. They are also extremely far away. Astronomers describe distance for objects like these with a number called **redshift** (written z): because the universe is expanding, light from a distant object gets stretched to longer, redder wavelengths on its way to us, and z measures how much it got stretched. These galaxies are at $z = 2$ to 5 , which means the light we're collecting today left them roughly 10 to 12.5 billion years ago. We are looking at the young universe.

Here's the interesting problem. Galaxies like this are violent places. Gas falls *into* them (feeding the star formation) and gas gets blasted *out* of them by exploding stars and radiation (shutting the star formation down). This push-and-pull, called **feedback**, is one of the big open questions in how galaxies grow. But it's very hard to see directly at these distances.

The trick astronomers use is to look for specific molecules in front of the galaxy. This project focuses on two "water-like" molecules that have lost an electron and become electrically charged: OH^+ ("O-H plus") and H_2O^+ ("water plus"). These are called **hydrides** — simple molecules made of one heavy atom plus hydrogen. They're useful because they only survive in particular conditions, so finding them tells you something specific about the gas. Each molecule absorbs light at one exact, known frequency — like a specific note on a piano. When that gas sits in front of the galaxy's bright glow, it removes light at that frequency, punching a narrow dip into the galaxy's spectrum. A **spectrum** is just a plot of how much light you receive at each frequency; the dip is called an **absorption line**.

Now, the actual question this thesis asks. Your galaxy — you'll be working on the target called **J0604** — should show not one but *two* of these absorption dips, because the H_2O^+ molecule's ground state is split into two closely spaced pieces, each producing its own line. (And if you're lucky, an OH^+ line lands in your data too.) The question is beautifully simple to state:

Do those two absorption dips sit at exactly the same velocity as each other, or is one shifted relative to the other?

If they line up perfectly, the two lines are being made by the same blob of gas — one layer, one thing happening. If there is a consistent shift between them, then the two lines are coming from *different* layers of gas moving at different speeds — which would be direct evidence that you’re seeing separate physical components, perhaps one being blown outward and one sitting quietly near the stars.

Why this framing is clever, and worth understanding before you start: normally, to say anything about how fast gas is moving in a distant galaxy, you first need to know the galaxy’s own redshift very precisely — and if that number is a little wrong, your velocity is wrong too, and your whole result collapses. But if you only compare *two lines in the same galaxy to each other*, any error in the galaxy’s redshift shifts both lines by exactly the same amount and **cancels out of the difference**. You are measuring a difference, not an absolute. That makes the result unusually robust, and it’s the reason this project is a good one for a beginner: the main measurement is protected from the single most common way this kind of work goes wrong.

What you will actually have at the end. Not a vague “understanding” — concrete things:

1. A **spectrum plot** for J0604 (and eventually the other galaxies in the program) showing the measured absorption dips, with fitted curves over them.
2. **One headline number with an error bar**: the velocity difference between the two lines, in kilometers per second. Something like “ $\Delta v = 45 \pm 15$ km/s” (the two lines are offset — separate gas layers) or “ $\Delta v = 0 \pm 12$ km/s” (no offset detectable, down to 12 km/s). *Both outcomes are real results*. A confident “no, they’re the same to within this precision” is genuinely publishable, and you should not feel you’ve failed if that’s what you get. This is important to internalize early, because beginners often panic when they don’t find a signal.
3. A **catalog table**: for every galaxy and every line, either a measurement or an honest upper limit (“if a dip this deep were there, we would have seen it — so it isn’t”).
4. A **completeness plot**: a demonstration, based on fake signals you inject into the real data, of how faint a dip you could actually have detected. This is what turns “I didn’t see anything” into a scientific statement instead of a shrug.
5. A **code repository** where every figure can be regenerated with one command.

0.2 2. Skills You’ll Need

None of this requires you to already be an astronomer. Here’s the honest list, with how deep you actually need to go.

Python — yes, essential, but at a modest level. You’ll use it for: reading data files, doing arithmetic on arrays of numbers, fitting a curve to a dip in a spectrum, making plots, and running the same procedure over 40 galaxies in a loop. You do *not* need to be a software engineer. What you need: variables, lists, loops, functions, ‘if’ statements, and reading error messages without giving up. Then four libraries, learned as you go rather than up front — ‘numpy’ (arrays of numbers and math on them), ‘matplotlib’ (plots), ‘astropy’ (astronomy-specific file formats and unit conversions), and ‘scipy’ (curve fitting and statistics). *If you don’t have this*: any beginner Python course will do for the language basics — a couple of weeks of evenings. Then specifically look for tutorials on “numpy arrays” and the official Astropy tutorials, which are written for exactly this audience and use real astronomy data.

CASA — a specialized astronomy program you’ll drive, not write. CASA is the software package for radio telescope data. You’ll use it to turn raw telescope recordings into images and spectra. It has a Python interface, so it feels like Python with extra commands. You don’t need to

understand its internals — you need to be able to run a handful of tasks (‘listobs’ to read what’s in a file, ‘tclean’ to make an image, ‘uvcontsub’ to subtract the background glow) and read their output. *If you don’t have this:* NRAO publishes CASA guides — step-by-step walkthroughs on practice datasets. Do one end-to-end before touching real data. Expect it to feel opaque for the first week; that’s normal and it passes.

The command line and Git — small but non-negotiable. Navigating folders, running scripts, copying files. Git for saving versions of your code so you can undo mistakes and so your final thesis is reproducible. A single afternoon tutorial on each covers what you need.

Astronomy concepts — five ideas, all learnable in a week of reading.

- *What a galaxy is*, and what makes a **starburst** unusual (star formation rate hundreds of times the normal rate).
- *Redshift* — light stretched by cosmic expansion; higher z means further away and further back in time. You need to understand the formula ‘observed frequency = rest frequency / (1 + z)’ and be able to use it. That’s the single most important equation in the whole project, and it’s just division.
- *The Doppler shift* — motion toward you compresses light to higher frequency, motion away stretches it to lower. This is how a frequency measurement becomes a velocity measurement. Same physics as a passing ambulance siren.
- *Emission vs. absorption lines* — molecules can either glow at their special frequency (emission, a bump) or block light at it (absorption, a dip). This project hunts dips.
- *Flux and continuum* — **flux** is simply how much energy per second you receive from the source. The **continuum** is the smooth, broad glow of the galaxy (mostly warm dust) with no lines in it. The absorption dips sit *on top of* this continuum, and you measure their depth relative to it. Because you always measure the dip relative to the local continuum, you never need to know the galaxy’s absolute brightness perfectly — another built-in robustness.

If you don’t have this: one chapter of an introductory astronomy textbook on spectra and the Doppler effect, plus a chapter on galaxies. Wikipedia is genuinely fine for redshift and Doppler shift. Ask your advisor to sketch the geometry (bright galaxy behind, gas cloud in front) on paper — five minutes of that beats an hour of reading.

Math — less than you’d fear. No calculus is required to do the work. You need: algebra (rearranging that redshift formula), what a logarithm is (the “optical depth” of a dip is defined with a natural log — it’s just a convenient way to express “how much light got removed”), and what a Gaussian curve looks like (the bell-shaped curve you’ll fit to each dip; its two key numbers are its *center* and its *width*, usually quoted as FWHM, “full width at half maximum” — how wide the bell is halfway down).

Statistics — the part beginners underestimate, so budget time for it. The whole project lives or dies on “is this dip real, or is it noise?” You need:

- *Noise and signal-to-noise* — every measurement wobbles; a feature is only believable if it’s much bigger than the wobble.
- *Error bars and how to combine them* — including a weighted average, where more precise measurements count more.

- *Bootstrap and permutation tests* — two beautiful “shuffle the data and see what happens by chance” methods that let you get honest uncertainties without any hard math. A permutation test is literally: randomly flip the signs of your measurements thousands of times, and see how often pure chance produces something as impressive as what you actually found. If chance does it often, you have nothing.
- Later, *MCMC* — a fancier fitting method. You can start with simple curve fitting and learn MCMC in month three; don’t let it block you now.

If you don’t have this: an introductory stats course covers the first two. For bootstrap and permutation tests, look for tutorials aimed at data scientists rather than mathematicians — they’re taught as recipes, and the recipes are short.

One habit that matters more than any skill: write down every number you use and where it came from. Which version of CASA. Which catalog you got the molecule’s rest frequency from and on what date. Which redshift you used and why. This project’s whole credibility rests on frequency precision, and a single unlabeled number can quietly poison a year of work.

0.3 3. Your First Week

The goal of Week 1 is **not** to detect anything. It is to answer one question: *for J0604, are the lines we care about even inside the data we have?* That question is the fork in the road for the entire thesis, and you can answer it in five days. Resist every urge to start hunting for signals.

Day 1 — Read, draw, and find the data.

Morning: read the proposal abstract and the research idea again, slowly, and every time you hit a word you can’t define, write it on a list. Then go define them. Expect twenty words. That’s fine.

Then draw the picture on paper: a bright galaxy on the left, a cloud of gas between it and you, the telescope on the right, and a spectrum below showing a flat line with two small dips in it. Label which dip is which molecule. Keep this sketch on your desk all year. Almost every confusion you’ll have this year is really a confusion about this picture.

Afternoon: go to the ALMA Science Archive website and search for the project code **2023.1.00804.S**. Don’t download anything yet — just look. Find J0604 in the target list. Look at how many files there are and how big they are (likely tens to hundreds of gigabytes for the full program). Check whether the data are public or still proprietary; if J0604 is still locked, that is not a disaster, it’s information — tell your advisor immediately, because there’s a well-defined fallback (build and test your entire analysis on realistic *fake* data with the same properties, then run it the day the real data unlock).

Also today: sort out where you’ll put the data. A laptop hard drive will not survive this project. Find out whether you have a university computing cluster account, and start that request now, because it takes days.

Realistic end of day: you understand the picture, you’ve found the data, you’ve started an account request. You have written no code. That’s a good Day 1.

Day 2 — Install everything and lose most of the day to it.

Install CASA (it’s a large download and it is fussy — on some systems it wants specific libraries), and set up a Python environment with numpy, scipy, matplotlib, and astropy. Make a git repository

for the project and commit an empty README.

Then start the download of *just J0604's* delivery in the background. One galaxy. Not forty. You want to learn the whole workflow on one target before you scale up.

Be prepared: **installation days are famously demoralizing**. You will hit an error that has nothing to do with astronomy. This is normal, it happens to professionals, and the fix is usually a five-minute search away. Budget the whole day for it and treat finishing the install as the day's success.

If the download is going to run overnight, spend the leftover time on a CASA guide tutorial using their small practice dataset. Getting one image out of practice data today will save you two days of confusion later.

Day 3 — Open the file and find out what's actually in it.

This is the day it starts feeling real. Run 'listobs' on J0604's measurement set (the file holding the telescope's raw recordings). You'll get a long text dump. Don't be intimidated — you're looking for one specific thing: the list of **spectral windows**.

A spectral window (or "spw") is simply a chunk of frequency the telescope was tuned to record — like a radio receiver that can listen to four separate stations at once instead of the whole dial. For each spw, write down: its ID number, its starting and ending frequency, how many channels it's chopped into, and how wide each channel is.

Then find the one detail that trips up almost everyone: the **frequency frame**. Frequencies can be quoted relative to the moving telescope on the spinning Earth ("TOPO") or corrected to a standard reference ("LSRK"). These differ by a fraction of a percent — invisible if you're careless, and fatal for a project whose entire result is a small frequency difference. Read which frame is stated in the file. Never assume. Write it down.

Build this into a spreadsheet or a small CSV file — one row per spectral window. This is "Table A," and you will use it all year.

Expect to get stuck today on the sheer volume of unfamiliar output. Ask your advisor to sit with you for fifteen minutes and point at the four lines in the 'listobs' output that matter. That's a far better use of their time than a general "how does ALMA work" conversation.

Day 4 — Look up the molecules.

Today you get the other half of the puzzle: the frequencies the molecules would emit if they were sitting still in a laboratory. These are called **rest frequencies**, and they're measured constants, not something you derive.

Go to Splatalogue (a searchable database of molecular line frequencies, drawing on the CDMS and JPL catalogs) and look up the ground-state transitions of H_2O^+ and OH^+ . Record, for each: the exact frequency, its quoted uncertainty, the quantum numbers, which catalog it came from, and today's date. That's "Table B."

Here's the wrinkle you'll hit, and it's worth understanding rather than skipping: each of these transitions isn't a single clean frequency. It's split into several closely spaced sub-lines (**hyperfine structure** — tiny energy splittings caused by the atomic nuclei). In a distant galaxy, gas moves fast enough that these sub-lines smear together into one broad dip, so you can't separate them. You therefore need to decide on *one* representative frequency per transition, and the sensible choice

is an average of the sub-lines weighted by how strong each one is. Write a tiny Python script that computes this average from the catalog values — ten lines of code. Do not do it by hand on a napkin, because the consistency of this convention across all your lines is *exactly* what your final measurement depends on.

If this feels fiddly and unimportant, that’s precisely the moment to slow down. It’s the most common place a project like this quietly goes wrong.

Day 5 — The coverage check: the day you find out what your thesis is.

Now combine the two tables. Take each rest frequency from Table B, apply the redshift formula using J0604’s approximate redshift (the value the proposal team used when they tuned the telescope — you’ll find it in the delivery itself), and get the predicted observed frequency:

$$> \text{observed frequency} = \text{rest frequency} \div (1 + z)$$

Then ask: does that predicted frequency land inside one of J0604’s spectral windows from Table A?

Three possible answers per line: **covered** (comfortably inside a window), **edge** (technically inside, but so close to the window’s edge that part of the dip would run off the end — unusable for measuring a center or a width), or **not covered**.

Two cautions. First, treat that redshift as approximate — it’s a bookkeeping number used to point the telescope, not a precise measurement. Its only job is to tell you *roughly where to look*. Never let it into a final velocity claim. Second, when you’re deciding “covered” vs. “edge,” remember the dips could be hundreds of km/s wide, so demand real margin from the window edge rather than accepting a bare landing inside.

Make this into a simple grid: galaxies down the side, molecular transitions across the top, colored by covered/edge/not covered. For now you only have one row (J0604). This grid is your first thesis figure, and once you extend it to all 40 galaxies it decides your path: if lots of galaxies have *two* lines covered, you do the main project (comparing the two lines). If almost none do, you do the equally valid backup project (a careful survey of how often these dips appear at all, and how deep they’d have to be for you to have seen them).

Days 6–7 (or the following Monday) — Consolidate, and make one ugly plot.

Don’t start anything new. Instead:

- Write up what you found in a page or two of plain English: the spectral windows, the frequency frame, the rest frequencies and where they came from, and the coverage answer for J0604. Send it to your advisor. Getting into a weekly written-update habit in Week 1 is worth more than any single technical skill.
- Commit everything to git — the tables, the hyperfine-averaging script, the notes.
- If time remains and the data are in hand, extract a rough spectrum from J0604 and plot it. It will look bad. The vertical scale will be wrong, the smooth galaxy glow won’t be subtracted, and you probably won’t see anything convincing at your predicted frequency. **Plot it anyway, and draw a vertical line at the predicted frequency.** Seeing your own data on your own screen, however crude, is what converts this from an abstract assignment into your project.

What counts as a good Week 1. You’ve installed the software, opened one galaxy’s data, read its spectral windows and frequency frame, recorded the molecular rest frequencies with a documented averaging convention, and answered “are the lines covered for J0604?” You may also have one ugly plot. You have detected nothing, and that’s expected — detections are a Month-3 activity, not a Week-1 one.

Where you’ll most likely get stuck, in order of probability: the CASA installation (Day 2); the sheer size and slowness of the data transfer (all week — start downloads before you need them and always test on one galaxy first); the TOPO-vs-LSRK frequency frame question (Day 3, and it will resurface); and the hyperfine averaging (Day 4). Every one of these is a known, solved problem. A fifteen-minute conversation with your advisor or a more senior student will unstick you faster than three hours of solo searching — and asking early is a sign of competence, not the opposite.